T-Adaptive Segment Routing

A scalar maximum-link-utilization model on Set A

Vuong Minh | October 2026 | Orange / EURO-ROADEF 2026

Abstract

This report studies the course Orange project derived from the EURO/ROADEF 2026 T-Adaptive Segment Routing challenge [6]. We implement and solve the scalar problem $\min L = \max_{t,a} \lambda(a,t)$, that is, the minimization of maximum link utilization (MLU). MLU is the first component of the challenge's lexicographic objective and a standard traffic-engineering criterion used to suppress hot spots and preserve residual capacity. All Set A instances are solved for period 0 (1800 s) and for periods 0 and 1 with the reconfiguration budget (3600 s), with averages over ten seeded runs. Bounds, gaps and the label "MLU-optimal" refer only to this scalar problem, not to full lexicographic optimality.

1 Introduction

Orange routes origin–destination demands on a directed IP/MPLS backbone. Between routing changes, traffic follows shortest paths induced by administrative metrics, and equal-cost paths share the flow. Segment Routing [3] inserts a short list of waypoints; between successive waypoints the network still uses shortest-path forwarding, so a compact header can move traffic away from a congested link.

The EURO/ROADEF 2026 challenge [6] defines T-Adaptive Segment Routing (T-ASR) over a general horizon $T = \{0, \dots, h-1\}$. In the course experiments we take $h = 2$: period 0 is the nominal topology, and period 1 withdraws some links for maintenance while the traffic matrix changes. A budget limits how many segments may change between the two periods.

The challenge objective [6] sorts every arc-period load and minimizes that vector lexicographically. This report solves only the primary scalar stage of that order: the maximum link utilization. Section 3 states the scientific rationale for that choice. We solve all twenty Set A instances on period 0 with a limit of 1800 s, then on both periods with the budget and a limit of 3600 s. Every table entry is a mean over ten runs with seeds 2026–2035.

2 Data and assumptions

Each Set A instance is given by three JSON files. The network file provides vertices, directed links, administrative metrics ω_a and capacities c_a. The traffic file provides, for every demand d, a source $s(d)$, a target $t(d)$ and a volume v_{dt} in each time slot. The scenario file gives maxSeg $= 6$, the period-1 budget κ and the links removed at period 1. Period 0 has no intervention and no budget. Identifiers are stored as given; they are not assumed to form an interval, and no Set A instance contains two links with the same ordered pair of endpoints.

For a period t, let $G(t) = (V, A(t))$ be the directed graph of available links. Dijkstra's algorithm from every vertex returns the distance $d_{ij}(t)$ and the number $\sigma_{ij}(t)$ of shortest directed i–j paths (path counts are stored in logarithmic form). For an arc $a = (u, v)$, the course note [1] defines the fraction of those paths that use a:

$$r_{ijat} = \begin{cases} \dfrac{\sigma_{iu}(t)\,\sigma_{vj}(t)}{\sigma_{ij}(t)}, & \text{if } d_{iu}(t) + \omega_a + d_{vj}(t) = d_{ij}(t), \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

This uniform-over-path formula is the one used throughout the experiments. It is not identical to a router-local ECMP recursion, which can assign different probabilities to complete shortest paths on an asymmetric DAG; loads are therefore comparable within this report, not necessarily to an official challenge evaluator that uses the other rule. If j is unreachable from i, the segment (i, j) is omitted.

3 Scalar MLU formulation

3.1 Objective and scientific rationale

Let $\lambda(a,t)$ be the load of arc a at period t, that is, the ratio of carried flow to capacity. The implemented objective is

$$\min\ L \qquad \text{with} \qquad L = \max_{t\in T,\, a\in A(t)} \lambda(a,t). \tag{2}$$

Minimizing maximum utilization is a classical intra-domain traffic-engineering objective: it pushes traffic away from bottlenecks, balances load across the topology and maximizes residual capacity for growth, bursts and failures [4, 5]. Queuing delay grows nonlinearly with utilization, so reducing the peak load improves the worst-case delay bound more directly than reducing the average load alone [4].

The challenge objective [6] extends (2) by continuing, after the peak is fixed, to the second-largest load, then the third, and so on. Equation (2) therefore determines the first component of that lexicographic order, but does not optimize the remaining components. All lower bounds, gaps and "MLU-optimal" labels below refer exclusively to (2).

3.2 Variables and constraints

The binary variable x_{dtij} equals one when demand d uses the segment from i to j in period t. On each period graph the model is

$$\sum_{j\neq i} x_{dtij} - \sum_{j\neq i} x_{dtji} = \begin{cases} 1, & i = s(d), \\ -1, & i = t(d), \\ 0, & \text{otherwise}, \end{cases} \qquad \forall\, d, i, \tag{3}$$

$$\sum_{i\neq j} x_{dtij} \le \text{maxSeg}, \qquad \forall\, d, \tag{4}$$

$$\sum_{d\in D} \sum_{i\neq j} r_{ijat}\, v_{dt}\, x_{dtij} \le c_a L, \qquad \forall\, a \in A(t). \tag{5}$$

Flow conservation on the segment graph produces an s–t chain. Degree inequalities (at most one selected segment leaving or entering a vertex, none leaving the target or entering the source) are added as valid cuts so that the source–target component is a simple path. A detached directed cycle remains formally possible; deleting it never increases L, because every segment contribution is nonnegative. Constraint (4) limits the header length; constraint (5) links ECMP load to L.

For the two-period model, (3)–(5) are written at $t = 0$ and $t = 1$. The period-1 graph deletes the intervention links, and both (1) and the volumes are recomputed. The symmetric difference of the two segment sets is linearized by $y_{dij} \ge 0$:

$$\begin{aligned} &y_{dij} \ge x_{d,1,ij} - x_{d,0,ij}, \quad y_{dij} \ge x_{d,0,ij} - x_{d,1,ij}, \\ &\sum_{d,i,j} y_{dij} \le \kappa. \end{aligned} \tag{6}$$

The direct routing $(s(d), t(d))$ in both periods has distance zero whenever both endpoints remain reachable, so a feasible budget-compliant solution always exists on the tested Set A instances.

4 Solution method

The model is written in Julia with JuMP 1.32.1 [2] and HiGHS 1.26.0, using one solver thread. The recorded time is the elapsed wall-clock time spent inside `optimize!`, summed over the solver calls of a run; graph construction and coefficient generation lie outside that timer.

If $|D|\,|T|\,|V|\,(|V|-1) \le 100\,000$, JuMP builds the compact MIP and HiGHS solves it directly (setA-01–03 in both experiments). Larger instances use column generation: each column is a segment path for one demand, the restricted master minimizes L, and pricing seeks a path of at most maxSeg hops with negative reduced cost. When the ordinary shortest-path weight is already nonnegative, no hop-limited path has negative reduced cost, so the corrected master bound is valid for the linear relaxation. Generated columns are then made binary and HiGHS applies branch-and-bound in the remaining time.

For two periods, each period is priced separately. Because the budget only removes solutions, a certified single-period MLU bound remains a lower bound on the joint maximum. A candidate is accepted only after an independent check of terminals, segment limit, reachability, repeated vertices, budget and recomputed maximum load. A run is labelled MLU-optimal when the checked value meets the bound; otherwise it is not certified. The gap is $1 - \mathrm{LB}/\mathrm{UB}$, and a zero gap is written only when the bound certifies the incumbent for (2).

5 Experimental protocol

Set A ranges from 20 vertices, 80 links and 40 demands to 400 vertices, 2000 links and 6000 demands. Limits are 1800 s for period 0 and 3600 s for the two-period model. A run may return early when the MLU bound certifies the incumbent. In Tables 1 and 2, MLU, `optimize!` time, gap and branch-and-bound nodes are means over ten runs. Status is categorical: *MLU-opt.* if all ten runs are certified for (2), and *Not cert.* otherwise (including mixed outcomes). An `optimize!` mean can slightly exceed the configured limit because a call is recorded only when it returns.

6 Results

Table 1: Period 0. Limit 1800 s. Means over ten runs. Status refers to scalar MLU (2).

Instance	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	MLU	Time (s)	Gap	Nodes	Status
setA-01	20	80	40	0.9294	1.05	0	1	MLU-opt.
setA-02	30	150	45	0.5486	2.79	0	1	MLU-opt.
setA-03	50	250	20	0.9435	39.43	0	1	MLU-opt.
setA-04	50	250	200	0.4591	1744.0	1.12×10^{-4}	105.7	Not cert.
setA-05	100	396	100	0.1155	0.03	0	0	MLU-opt.
setA-06	100	500	500	0.0986	352.3	0	795	MLU-opt.
setA-07	100	500	800	0.9080	0.03	0	0	MLU-opt.
setA-08	150	654	200	0.3189	2.69	0	2.1	MLU-opt.
setA-09	150	750	200	0.5054	0.04	0	1	MLU-opt.
setA-10	150	966	1000	0.0761	1739.3	0.1020	272.6	Not cert.
setA-11	200	1000	400	0.6555	1741.0	0.0646	1.3	Not cert.
setA-12	200	898	400	0.6401	1299.3	0.0955	9.9	Not cert.
setA-13	200	1000	2000	0.0389	1741.5	9.99×10^{-4}	2690.6	Not cert.
setA-14	250	1108	600	0.4240	1745.2	1.43×10^{-4}	119.2	Not cert.
setA-15	250	1250	600	0.8252	0.04	0	0	MLU-opt.
setA-16	250	1452	4800	0.0393	1787.0	0.0172	12398.5	Not cert.
setA-17	300	1270	2000	0.4242	0.05	0	0	MLU-opt.
setA-18	300	1500	2000	0.6334	0.04	0	0	MLU-opt.
setA-19	300	1998	6000	0.0483	1774.2	0.0296	8516.1	Not cert.
setA-20	400	2000	6000	0.9913	0.09	0	0	MLU-opt.

On period 0, twelve of the twenty instances are MLU-optimal. Several certificates arrive in well under a second, including setA-20 ($|V| = 400$, $|D| = 6000$), where the column-generation bound meets the incumbent at the root. The other eight instances consume essentially the full 1800 s budget. setA-04, setA-13 and setA-14 stop with a gap below 10^{-3}; setA-10–12 remain between 6% and 10% above the bound.

Table 2: Periods 0 and 1 with budget. Limit 3600 s. Status refers to scalar MLU (2).

Instance	$\|V\|$	$\|A\|$	$\|D\|$	MLU	Time (s)	Gap	Nodes	Status
setA-01	20	80	40	0.9294	14.61	0	1	MLU-opt.
setA-02	30	150	45	0.9031	152.7	0	1	MLU-opt.
setA-03[a]	50	250	20	0.9437	1430.5	0.0030	33.1	Not cert.[a]
setA-04	50	250	200	0.5812	3592.7	1.61×10^{-4}	1503.4	Not cert.
setA-05	100	396	100	0.2050	0.32	0	1	MLU-opt.
setA-06	100	500	500	0.0986	168.0	0	540.8	MLU-opt.
setA-07	100	500	800	0.9080	42.32	0	229.6	MLU-opt.
setA-08	150	654	200	0.3189	1.26	0	3	MLU-opt.
setA-09	150	750	200	0.8497	2.60	0	2	MLU-opt.
setA-10	150	966	1000	0.0733	3594.1	0.0678	4947	Not cert.
setA-11	200	1000	400	0.7858	2.92	0	2	MLU-opt.
setA-12	200	898	400	0.8799	1.82	0	2	MLU-opt.
setA-13	200	1000	2000	0.0420	3591.1	0.0288	7846.2	Not cert.
setA-14	250	1108	600	0.5176	3608.9	0.0197	17922.4	Not cert.
setA-15	250	1250	600	0.8987	5.61	0	1	MLU-opt.
setA-16	250	1452	4800	0.1369	3594.2	0.6682	32238.2	Not cert.
setA-17	300	1270	2000	0.4242	50.51	0	1	MLU-opt.
setA-18	300	1500	2000	1.0000	32.79	0	0	MLU-opt.
setA-19	300	1998	6000	0.1096	3542.7	0.5586	15254	Not cert.
setA-20	400	2000	6000	0.9913	401.9	0	2	MLU-opt.

[a] Eight runs are MLU-optimal at 0.9435; two stop at 0.9444.

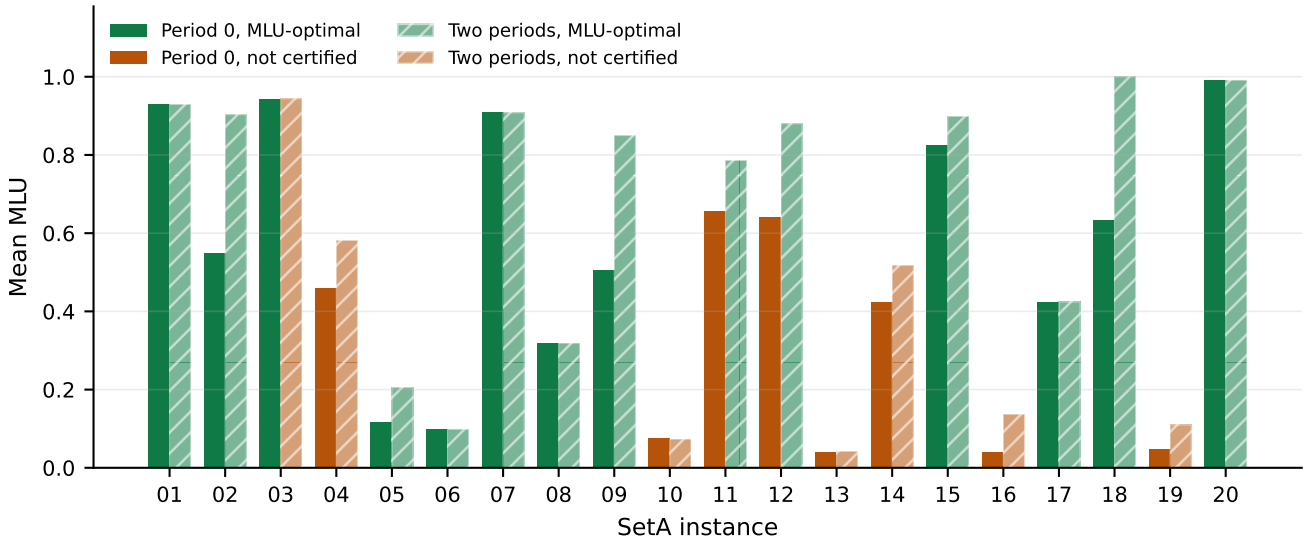


Figure 1: Mean MLU on period 0 (solid bars) and on both periods (hatched). Green: all ten runs certified for (2); amber: not certified.

Thirteen two-period instances are MLU-optimal. Adding period 1 does not change the reported MLU of setA-01, 06, 07, 08, 17 and 20: a feasible routing meets the budget without raising the peak load (for the certified rows among them, this equality is proven). Elsewhere the second period raises the peak: setA-02 from 0.5486 to 0.9031, setA-05 from 0.1155 to 0.2050, setA-09 from 0.5054 to 0.8497, and setA-18 from 0.6334 to 1. setA-11 and setA-12 are not certified on period 0, yet their higher two-period values 0.7858 and 0.8799 are certified. setA-03 is mixed: eight runs prove 0.9435 and two stop at 0.9444.

7 Discussion and limitations

The weakest two-period results are setA-16 and setA-19, with mean gaps 0.67 and 0.56 after one hour. The best feasible value among the ten runs is better than the reported mean: 0.1025 for setA-16 and 0.0796 for setA-19. A missing MLU certificate stays “Not cert.”, and a bound that was not computed is never written as zero.

Two modelling choices delimit the scope of the tables. First, the formulation optimizes (2), not the full lexicographic vector of loads [6]; two routings with the same peak may still differ on the remaining sorted loads. Second, the split coefficients follow the course note [1] rather than a router-local ECMP recursion. Within those choices, every row comes from the MIP above, either compact or as a path formulation with a computed lower bound on L.

8 Conclusion

We formulated and solved a scalar MLU model for the course Orange project on all Set A instances, in both the period-0 and the two-period settings. MLU is the scientifically standard primary traffic-engineering criterion and the first term of the challenge lexicographic objective; the report states that limitation explicitly. Twelve period-0 instances and thirteen two-period instances are certified for (2) within the prescribed time limits. Future work could continue the lexicographic sequence on instances that certify L early, or replace (1) by router-local ECMP if official challenge comparability is required.

References

[1] Course note. Computing arc-use ratios on shortest paths, 2026.

[2] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017.

[3] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir. Segment routing architecture. IETF RFC 8402, 2018.

[4] Bernard Fortz, Jennifer Rexford, and Mikkel Thorup. Traffic engineering with traditional IP routing protocols. *IEEE Communications Magazine*, 40(10):118–124, 2002.

[5] Srikanth Kandula, Dina Katabi, Bruce Davie, and Anna Charny. Walking the tightrope: Responsive yet stable traffic engineering. In *Proceedings of ACM SIGCOMM*, pages 253–264, 2005.

[6] Orange Research. Challenge EURO/ROADEF 2026 problem description, 2026.